Notre architecture agentique

Groupe 3 — Projet B (emplois du temps), approche agentique

Version de travail — novembre 2026

L'équipe n'a pas un outillage homogène : une partie utilise Claude Code, une autre d'autres outils. Ce document décrit d'abord ce qui est commun à tous, puis chaque configuration individuelle.

1 Socle commun à l'équipe

Élément	**Rôle**
`AGENTS.md` (versionné)	Contexte lu par l'agent à chaque session : stack, conventions de code, règles de sécurité, ce qu'il faut demander avant de faire.
Branches et PR	`feature/*` → `dev` → `main`. Tout passe par une PR relue par un humain.
Trailers de commit	`IA: genere` / `IA: retouche` / rien : origine de chaque ligne de code, pour mesurer les retouches et les bugs.
Journal `journal/ia.csv`	Une ligne par tâche : temps, interventions humaines, tokens, coût. Écrite par l'agent en fin de tâche.
Tests	pytest sur le vrai PostgreSQL. CI à venir.

2 Daniel : Claude Code

2.1 Composants

Composant	Configuration	Utilisé ?
Agent principal	Claude Code, extension VS Code	oui
Modèle	Claude Opus 5.5, effort moyen, fixé pour le projet : adapté à un gros projet, avec des tâches longues et beaucoup de composants à prendre en compte. Sonnet 5.5 pour les questions hors code.	oui
Autonomie	L'agent modifie les fichiers et lance les commandes sans demander de confirmation. Seuls le commit et le push attendent l'accord de l'humain.	oui
Contexte projet	`AGENTS.md` (commun)	oui
Contexte personnel	Fichier global : langue, réponses courtes, environnement (Ubuntu, GPU), commandes à donner sur une ligne	oui
Outils web	Recherche et lecture de pages, pour vérifier les sources du rapport	oui
Skills	`claude-api` (tarifs des modèles pour le calcul du coût) ; 25 skills d'ingénierie du plugin agent-skills [1] (spec, plan, tests, revue…)	1 sur 26
Sous-agents	Intégrés : exploration, planification. Plugin : relecteur de code, auditeur sécurité, ingénieur test, auditeur performance web [2]	non
Hooks	Un hook personnel au démarrage (réseau utilisé) ; aucun hook propre au projet	non (projet)
Mémoire persistante	Disponible, vide pour ce projet	non

2.2 Boucle de travail

mise en contexte (documents, décisions, contraintes) → pour chaque tâche, une par une : l'agent exécute → **vérification et validation humaines** → commit et push **après accord** → PR relue → fusion dans `dev` → ligne de journal

Les tâches sont confiées une à la fois plutôt qu'en un seul gros prompt : chaque résultat est vérifié avant de passer à la suite. Les actions sensibles (suppression de données, ajout de dépendance, modification du socle commun) demandent une confirmation, comme le prévoit `AGENTS.md`. L'agent ne pousse jamais directement sur `main` ou `dev`.

2.3 Ce qui est délégué, ce qui reste humain

Délégué à l'agent	Fait ou décidé par l'humain
Schéma de base et migrations	Relecture des migrations (perte de données, contraintes)
Générateur de jeux de données	Validation de la cohérence des données générées
Implémentation de l'algorithme de génération	Conception de l'algorithme : modélisation, heuristiques, complexité
Tests, scripts de mesure, documentation	Choix d'architecture, priorités, validation de chaque tâche

2.4 À évaluer ensuite

- **Sous-agent relecteur** avant chaque PR : est-ce qu'il réduit le taux de retouche et les bugs ?

- **Sous-agent ingénieur test** sur le backend : est-ce qu'il améliore la couverture et le taux de mutation ?
- **Hook projet** lançant les tests avant un commit : moins de corrections après coup ?
- Chaque ajout est noté `infra-agent` dans le journal, pour mesurer son coût de mise en place.

3 Thomas : Claude Code

3.1 Composants

Composant	Configuration	Utilisé ?
Agent principal	Claude Code et parfois Opencode sous Windows 11 (interface : Terminal Powershell/TUI)	oui
Modèle	Claude Opus 5.5 par défaut, effort élevé. Raison du choix : Modèle le plus performant aujourd'hui (Nov. 2026)	oui
Autonomie	Mode `auto` : l'agent modifie les fichiers et lance les commandes sans demander, les actions risquées restent soumises à confirmation. Le commit n'est fait que sur demande explicite (fichier global).	oui
Contexte projet	`AGENTS.md` (commun)	oui
Contexte personnel	Fichier global : priorités (exactitude, preuves, sécurité, changements minimaux), interdits (inventer un chemin ou un résultat de test, affaiblir un test, exposer un secret), discipline de code (pas de code mort, pas d'abstraction spéculative), tests sans mocks, commits en Conventional Commits. Réponses en français.	oui
Outils web	Recherche web ; extension Claude in Chrome/Browser Use (pilotage du navigateur)	oui
Serveurs MCP	`codegraph` (graphe de symboles du code) / connecteurs Claude.ai	oui
Skills	`karpathy-guidelines` (garde-fous contre les erreurs courantes des LLM) ; plugin superpowers (obra/superpowers, v6.4.2) : brainstorming, plans, TDD, débogage systématique, vérification avant de conclure, revue de code, développement par sous-agents…	oui
Sous-agents	Orchestrator qui appelle des sous agents : explorer, librarian, oracle, designer et fixer.	parfois, selon la tâche

3.2 Boucle de travail

Dans le prompt : Idée de feature/correction d'un bug provenant d'une issue ou autre source → l'agent propose un brainstorm puis specs puis plan → je valide le plan → l'agent exécute le plan → l'agent test ses implémentations → relecture du code rapide -> commit et push **après accord** → PR relue → fusion dans `dev` → ligne de journal

3.3 Ce qui est délégué, ce qui reste humain

Délégué à l'agent	Fait ou décidé par l'humain
Ecriture de code	Relécture du code et validation de la cohérence des choix

3.4 À évaluer ensuite

Qu'est ce qui est le plus efficace/le plus cher : sous agent ou pas ?

4 Autres membres

À compléter par chaque membre, une sous-section par personne : outil, modèle, contexte, ce qui est délégué à l'IA et ce qui reste manuel.

Références

[1] A. Osmani, « agent-skills: plugin Claude Code (sous-agents et skills d'ingénierie logicielle) ». [En ligne]. Disponible sur: https://github.com/addyosmani/agent-skills

[2] Anthropic, « Claude Code documentation: Subagents ». [En ligne]. Disponible sur: https://code.claude.com/docs/en/sub-agents